

CNN 활용 머신비전 통합 시스템 상세 프로젝트 기획서

음료/식품 용기 생산라인 적용
빈 용기 검사 → 캡/라벨 조립 검사

프로젝트명	Factory — Smart Bottle Line Vision QC
대상 제품	삼다수 페트병 (단일 제품 기준, 재학습으로 전환 가능)
프로젝트 기간	12일 (기획 3일 + 개발 8일 + 시연 1일)
기술 스택	Pylon(C++), Python(PyTorch/YOLO11), TCP/IP, MariaDB, MFC, Arduino
목적	취업 포트폴리오용 CNN 머신비전 프로젝트
버전	Ver. 0.12

목 차

목 차	2
1. 프로젝트 개요	4
1.1 프로젝트명	4
1.2 배경 및 목적	4
1.3 검사 대상	4
1.4 왜 CNN인가? — 기존 시스템 대비 경제적 이득	4
1.5 핵심 키워드	4
1.6 프로젝트 평가 지표	5
2. 시스템 구성	6
2.1 2단계 파이프라인 개요	6
2.2 전체 시스템 아키텍처	6
2.3 서버 분리 전략 (4-서버 구성)	6
3. 단계별 상세 설계	7
3.1 ① 입고 검사 — 빈 용기 외관 결함 탐지	7
3.1.1 목적	7
3.1.2 CNN 모델 설계	7
3.1.3 하드웨어 구성	7
3.2 ② 조립 검사 — 캡+라벨 조립 완성도 검사	7
3.2.1 목적	7
3.2.2 검사 항목	7
3.2.3 CNN 모델 설계 (하이브리드)	7
3.2.4 불량 시뮬레이션	8
4. 데이터 전략	9
4.1 데이터 수집 방침	9
4.2 직접 데이터 수집 계획	9
4.3 공개 데이터셋 채용 기준	9
4.4 데이터 증강 전략	9
5. 기술 스택 및 통신 프로토콜	10
5.1 명명 규칙 (Naming Convention)	10

5.2 기술 스택 상세	10
5.3 TCP/IP 통신 프로토콜	10
5.4 서버 헬스체크 (Health-Check)	10
6. 전체 시스템 흐름도	12
7. 작업 영역 요약	14
7.1 영역 간 주요 접점	15
8. GUI 설계 (통합 대시보드)	16
9. DB 스키마 (ERD)	17
9.1 테이블 관계 요약 (ERD)	18
10. 아두이노 연동 설계	19
11. 개발 일정 (12일)	20
11.1 병렬 개발 구조 (Day 4~9)	20
12. 상업성 분석 및 포트폴리오 어필	21
12.1 적용 분야	21
12.2 면접 활용 멘트	21
12.3 포트폴리오 어필 포인트	21
12.4 확장 가능성	21

1. 프로젝트 개요

1.1 프로젝트명

Factory — Smart Bottle Line Vision QC

CNN 기반 음료 용기 입고·조립 2단계 통합 품질관리 시스템

1.2 배경 및 목적

음료/식품 생산라인에서 빈 용기의 미세 결함, 캡/라벨 조립 불량은 품질 사고와 비용 증가로 직결됩니다. 특히 페트병의 이물질, 라벨 인쇄 번짐, 캡 기울어짐 같은 결함은 기존 룰 기반 머신비전(밝기 임계값, 치수 비교)으로 탐지하기 어렵습니다. 결함 형태가 불규칙하고 발생 위치가 매번 달라, 정상과 불량의 경계가 미묘하기 때문입니다.

본 프로젝트는 CNN 기반 이상탐지(PatchCore)와 객체탐지(YOLO11)를 결합하여, 정상 패턴 학습 후 미묘한 이상을 감지하는 방식으로 이 한계를 해결하고, 입고→조립 핵심 2개 공정을 통합 관리하는 시스템을 12일 내에 구축합니다.

1.3 검사 대상

항목	내용
대상 용기	삼다수 페트병 (단일 제품 기준)
검사 스테이션	① 입고 검사 (빈 용기), ② 조립 검사 (캡+라벨 완성품)
제품 전환 시	대상 제품 이미지로 AI 모델 재학습 필요 (클라이언트 UI에서 재학습 트리거 기능 구현 예정)

1.4 왜 CNN인가? — 기존 시스템 대비 경제적 이득

검사 항목	기존 룰 기반	CNN 기반	CNN이 유리한 이유
용기 내부 미세 이물질	투명/반투명 용기 투과 검사 시 임계값 설정 어려움	정상 패턴 학습 후 이상 감지	이물질 형태/위치/크기가 매번 다름
라벨 인쇄 불량	번짐/흐림의 정도가 다양해 규칙 정의 불가	정상 인쇄 패턴과 비교하여 이상 감지	불량 패턴이 불규칙·미묘하여 경계값 설정 어려움
캡 기울어짐/ 미체결	각도 임계값으로 잡을 수 있으나 오탐지 높음	정상 캡 상태를 학습하여 미세 차이 감지	조명 변화·병 형태 차이에 강건
다품종 전환	제품마다 룰 재설정 필요 → 비용 폭발	재학습만으로 적용 (룰 재설정 대비 저비용)	신제품 출시 시 정상 이미지 촬영 + 재학습으로 빠른 대응

1.5 핵심 키워드

구분	내용
기술	CNN, 이상탐지(PatchCore), 객체탐지(YOLO11)
산업	HACCP, 식품안전법, GMP, 스마트팩토리, 품질관리
기술 스택	Pylon SDK(C++), Python(PyTorch/Ultralytics), TCP/IP, MariaDB, MFC, Arduino

적용 분야	음료/식품/제약 생산라인, K-뷰티 화장품 포장라인
-------	------------------------------

1.6 프로젝트 평가 지표

데이터 수집 용이성	산업성	시연 효과	난이도
★★★★★	★★★★★	★★★★★	상

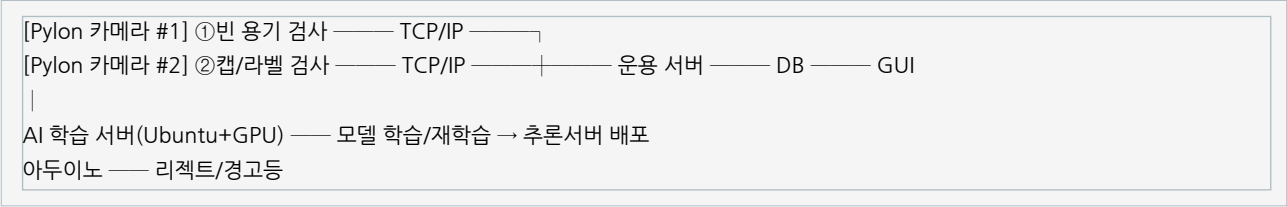
2. 시스템 구성

2.1 2단계 파이프라인 개요

실제 음료 공장 라인과 동일한 흐름으로 2개 비전 검사 스테이션을 배치합니다.

단계	공정명	AI/알고리즘	역할	판정 기준	출력 액션
① 입고 검사	빈 용기 외관 결함 탐지	PatchCore (이상탐지)	OK/NG (크랙/이물질/오염/파손)	이상점수 임계값 초과 시 NG	NG 용기 리젝트 DB 기록
② 조립 검사	캡+라벨 조립 완성도 검사	YOLO11 (구조) + PatchCore (표면)	캡 정상체결/라벨 정위치	YOLO11 IoU 판정 + PatchCore 이상점수	NG 제품 경고 재작업 지시

2.2 전체 시스템 아키텍처



2.3 서버 분리 전략 (4-서버 구성)

구분	역할	HW	OS / 비고
AI 학습 서버	모델 학습, 하이퍼파라미터 튜닝, 데이터 증강, 가중치 배포	GPU 서버 (NVIDIA RTX 계열)	Ubuntu 24.04 + CUDA 학습 완료 모델 → 추론서버 전송(SCP/공유폴더)
AI 추론 서버 #1 (입고 검사)	Pylon 카메라 #1 연결 PatchCore 추론 결과 → 운용서버 TCP 송신	Edge PC #1 (CPU/경량 GPU)	Ubuntu / Windows station1_runner.py 실행
AI 추론 서버 #2 (조립 검사)	Pylon 카메라 #2 연결 YOLO11 + PatchCore 추론 결과 → 운용서버 TCP 송신	Edge PC #2 (CPU/경량 GPU)	Ubuntu / Windows station2_runner.py 실행
운용 서버 (Main Server)	TCP 수신·라우팅, 결과 검증, 통계 집계, DB 기록, MFC GUI 푸시	일반 서버 (Windows)	Windows 10/11 Pylon SDK 호환 C++ (MFC + TCP)

※ 운용 서버는 AI 학습 서버·추론 서버의 생존 여부를 주기적으로 확인(Health-Check)합니다. 상세 내용은 5.4절을 참조하세요.

3. 단계별 상세 설계

3.1 ① 입고 검사 — 빈 용기 외관 결함 탐지

3.1.1 목적

생산라인에 투입되기 전, 빈 페트병(삼다수)의 외관 결함을 CNN 이상탐지로 판별합니다. 결함 용기가 충전 공정에 투입되면 내용물 오염, 파병 사고 등으로 직결되므로 사전 차단이 필수입니다. 용기는 재사용하지 않으므로 결과는 성공(정상)/실패(불량)로만 출력합니다.

3.1.2 CNN 모델 설계

항목	상세
접근 방식	이상탐지 (Anomaly Detection) — 정상 용기만으로 학습, 결함을 '정상과 다른 것'으로 탐지
적용 모델	PatchCore (이상탐지 SOTA) — 백본: 사전학습 ResNet 계열 특징 추출기
입력 크기	224×224 또는 256×256 (Pylon 최대 해상도에서 crop/resize)
출력값	성공(OK) / 실패(NG) + 결함 위치 히트맵 + 이상 점수(anomaly score)
학습 전략	정상 이미지만으로 학습 (비지도학습, unsupervised). 결함 데이터 부족 문제 해결
결과 전송	실패(NG) 시에만 운용서버로 결과값 + 로그 + 사진 전송. 성공(OK) 시 전송 없음.

3.1.3 하드웨어 구성

구성 요소	상세
카메라	Pylon 산업용 카메라 + 백라이트(투과 검사) + 확산조명(표면 검사)
아두이노	NG 시 서보모터 리젝트 + 빨간 LED + 부저 경고음

3.2 ② 조립 검사 — 캡+라벨 조립 완성도 검사

3.2.1 목적

①에서 합격한 용기에 내용물을 충전한 후, 캡 체결 + 라벨 부착이 정상적으로 완료되었는지 검사합니다. 실제 음료 공장에서는 캡 미체결, 라벨 기울어짐은 소비자 클레임의 주요 원인입니다.

3.2.2 검사 항목

검사 항목	정상	불량 예시
캡 체결	캡이 완전히 닫힘, 수평	캡 미체결, 기울어짐, 없음
라벨 부착	라벨이 정위치에 바르게 부착	라벨 기울어짐, 찢어짐, 없음
충전량	액면이 기준선 범위 안	과충전, 미충전

※ 씰링(sealing) 검사는 데이터 부족으로 제외합니다.

3.2.3 CNN 모델 설계 (하이브리드)

항목	상세
----	----

접근 방식	객체탐지(YOLO11) + 이상탐지(PatchCore) 하이브리드 — 구조적 결함은 객체탐지, 표면 품질 결함(번짐/불균일)은 이상탐지로 분담
객체탐지 모델	YOLO11 — 입력 640×640
이상탐지 모델	PatchCore — ROI crop 224×224 (라벨 표면 품질 검사)
YOLO11 탐지 대상	cap, label, liquid_level (각 요소의 존재 여부 + 위치 IoU 기반 정상/불량 판정)
YOLO11 출력	이상 클래스 및 바운딩 박스 오버레이 이미지 포함
판정 로직	1차: YOLO11로 각 요소 존재 + IoU 기반 위치 판정 2차: PatchCore로 라벨 표면 이상 점수 판정 → 두 판정 중 하나라도 NG이면 최종 NG
결과 전송	실패(NG) 시에만 운용서버로 결과값 + 로그 + 바운딩박스 오버레이 사진 전송. 성공(OK) 시 전송 없음.

3.2.4 불량 시뮬레이션

캡 안 닫기, 캡 빼기, 라벨 기울여 붙이기, 라벨 찢기, 물 높이 다르게 채우기 등으로 쉽게 연출 가능합니다.

4. 데이터 전략

4.1 데이터 수집 방침

직접 촬영 데이터를 기본으로 하되, 도메인이 유사한 공개 데이터셋이 확인될 경우 적극 채용합니다.

4.2 직접 데이터 수집 계획

단계	수집 방법	예상 량	불량 생성 방법
①입고	삼다수 페트병 Pylon 직접 촬영	200~300장	이물질 배치, 마커 낙서, 테이프 부착, 스크래치
②조립	음료병에 캡/라벨 조립 후 정상/불량 촬영	100~200장	캡 안 닫기, 라벨 기울이기, 물 높이 조절

4.3 공개 데이터셋 채용 기준

직접 수집 데이터만으로 학습량이 부족할 경우, 페트병/투명 용기 도메인과 유사한 공개 데이터셋을 적극 탐색·채용합니다. 채용 시 이 기획서에 내용을 업데이트합니다.

4.4 데이터 증강 전략

모든 단계 공통 적용: 회전(0/90/180/270도), 수평/수직 반전, 밝기/대비 조정, Gaussian Noise, Random Crop, CutOut/CutMix. 수집량 대비 5~10배 확장 목표.

5. 기술 스택 및 통신 프로토콜

5.1 명명 규칙 (Naming Convention)

대상	규칙	예시
파일명	파스칼 표기법 (PascalCase)	StationRunner.py, DbManager.cpp
클래스명	파스칼 표기법 (PascalCase)	StationInferencer, PacketBuilder
변수명	스네이크 표기법 (snake_case)	anomaly_score, result_dict
메소드명	스네이크 표기법 (snake_case)	build_packet(), infer_image()

5.2 기술 스택 상세

구분	기술	비고
카메라	Basler Pylon SDK (C++)	산업용 카메라 제어
이상탐지 AI	PyTorch + Anomalib (PatchCore)	모델 학습 + 추론
객체탐지 AI	PyTorch + Ultralytics (YOLO11)	모델 학습 + 추론
통신	TCP/IP Socket (Python/C++)	추론서버 ↔ 운용서버 이미지 전송
DB	MariaDB	검사 결과, 이력, 통계
GUI	MFC	2단계 통합 대시보드
아두이노	Arduino Uno/Mega + Serial	리젝트, 경고등

5.3 TCP/IP 통신 프로토콜

항목	상세
프로토콜	Custom TCP Socket (JSON 기반 메시지)
메시지 형식	[4byte 길이 헤더(JSON 크기)] + [JSON payload] + [Image binary (NG 시에만 포함)]
요청 예시	{"station": 1, "type": "inspect", "timestamp": "...", "image_size": 921600}
응답 예시 (NG)	{"station": 1, "result": "NG", "defect": "contamination", "score": 0.87, "action": "reject", "image": <binary>}
전송 조건	NG(실패) 시에만 결과값 + 로그 + 사진 전송. OK(성공) 시 전송 없음.
수신 흐름	① 4바이트 읽기 → JSON 크기 파악 ② 해당 크기만큼 읽기 → JSON 파싱, image_size 확인 ③ image_size만큼 바이너리 읽기 (NG 시) → 이미지 획득 ④ CNN 추론 → 결과 JSON 응답

5.4 서버 헬스체크 (Health-Check)

운용 서버(Main Server)는 AI 학습 서버, 추론 서버 #1, 추론 서버 #2의 생존 여부를 주기적으로 확인하는 헬스체크 프로세스를 실행합니다.

항목	내용
----	----

방식	TCP Heartbeat — 운용서버가 주기적으로 각 서버에 ping 패킷 전송, pong 수신 확인
주기	5초 간격 (설정 변경 가능)
타임아웃	3회 무응답 시 해당 서버 '장애' 상태로 판정
GUI 표시	MFC 대시보드 상단에 각 서버 상태 표시 (초록: 정상 / 빨강: 장애)
로그	장애 발생/복구 이벤트를 DB 및 로그 파일에 기록
알림	장애 발생 시 부저 또는 팝업 경고

6. 전체 시스템 흐름도

아래 흐름도는 학습 서버 → AI 추론 서버 → 운용 서버 → MFC 클라이언트의 전체 데이터 흐름을 나타냅니다.

[1] AI 학습 서버 (Training Server) — Ubuntu + GPU

```
ai/station1/Train.py → PatchCore (①입고)
ai/station2/TrainYolo.py → YOLO11 (②구조) + PatchCore (②표면)
↓
ai/models/*.ckpt, *.pt (학습된 가중치)
↓
SCP / 공유폴더 / USB 로 배포 → 추론 서버로 전송
```

[2-A] AI 추론 서버 #1 — ① 입고 검사 스테이션

```
[Pylon Camera #1] — GigE + Pylon SDK
↓
StationRunner.py (station1)
1. Pylon grab → 2. BGR 변환 → 3. infer() 호출
4. 결과 → 패킷 → 5. TCP 송신 (NG 시에만) → 6. Arduino 제어
↓ [AI 추론 접점]
ai/station1/Inference.py → Station1Inferencer.infer(image)
├─ PatchCore 추론
├─ return dict {result, score, defect, heatmap}
↓
build_packet() → NG: 결과값 + 로그 + 사진 포함
[Arduino Serial] → NG: 서보 리젝트 + LED + 부저
```

[2-B] AI 추론 서버 #2 — ② 조립 검사 스테이션

```
[Pylon Camera #2] — GigE + Pylon SDK
↓
StationRunner.py (station2)
1. Pylon grab → 2. BGR 변환 → 3. infer() 호출
4. 결과 → 패킷 → 5. TCP 송신 (NG 시에만) → 6. Arduino 제어
↓ [AI 추론 접점]
ai/station2/Inference.py → Station2Inferencer.infer(image)
├─ YOLO11 (640x640) → ROI crop 224
├─ PatchCore (라벨 표면 품질)
├─ Result Fusion
├─ return dict {result, score, defects[], detections[], bbox_overlay}
↓
build_packet() → NG: 결과값 + 바운딩박스 오버레이 사진 포함
[Arduino Serial] → NG: LCD + RGB LED
```

[3] 운용 서버 (Main Server, C++) — Windows

```

[TCP Listener :9000] — epoll/select
↓
recv 4byte length → recv JSON payload → recv image (NG만)
↓
[Router] — station 필드로 분기
├ Station1Handler — JSON 파싱 / 결과 객체 / defect 정리
└ Station2Handler — JSON 파싱 / 결과 객체 / defects[]
↓
[Manager] — 결과 검증 / 통계 집계 / NG 이미지 저장 판단
↓
├ DbManager → MariaDB (inspections/bottles/assemblies/models)
├ ImageStorage → ./storage/station1|2/YYYYMMDD/*.jpg
└ GuiNotifier → MFC 클라이언트 PostMessage 실시간 푸시
↓
[HealthCheck Thread] — 추론서버 #1, #2, 학습서버 생존 여부 주기 확인

```

[4] MFC Admin Client (C++) — Windows

```

[Background recv Thread] — PostMessage (thread-safe UI update)
↓
[종합현황] [① 입고] [② 조립] [통계/이력] [모델관리]
├ 실시간 영상 / OK·NG 표시 / 결함 히트맵
├ 불량률 차트 / 파레토 차트 / Latency 추이
└ 서버 헬스체크 상태 표시 (정상: 초록 / 장애: 빨강)

```

데이터 흐름 한 줄 요약:

Pylon → infer() → dict → JSON → TCP → Router → Handler → Manager → DB/Storage → GUI → 화면

7. 작업 영역 요약

본 시스템은 크게 5개 작업 영역으로 구성됩니다. 각 영역은 독립적으로 개발·테스트할 수 있으며, 영역 간 접점은 최소화하여 병렬 개발이 가능합니다.

AI 모델 학습 영역	
작업 항목	상세
입고 검사 모델 학습	PatchCore 학습 — ai/station1/Train.py
조립 검사 모델 학습 (구조)	YOLO11 학습 — ai/station2/TrainYolo.py
조립 검사 모델 학습 (표면)	PatchCore 학습 — ai/station2/TrainPatchcore.py
추론 인터페이스 구현	Station1Inferencer / Station2Inferencer.infer() — ai/*/Inference.py
모델 가중치 관리	ai/models/*.ckpt, *.pt — 버전 태깅 및 배포
환경 구축	CUDA, PyTorch, Anomalib, Ultralytics 설치 및 학습 환경 구성

AI 추론 서버 영역 (Edge PC)	
작업 항목	상세
카메라 수집	Pylon SDK 연동 — GigE grab, BGR 변환 — edge/StationRunner.py
추론 호출	Station1/2Inferencer.infer(image) 호출 및 결과 수신
패킷 빌드	NG 시 결과+로그+이미지 포함 패킷 생성 — edge/common/Packet.py
TCP 송신	운용 서버로 TCP 전송 (NG 시에만)
Arduino 제어	시리얼 통신으로 리젝트/경고 제어 — edge/common/SerialCtrl.py
설정 관리	서버 IP, 포트, 모델 경로 — edge/Config.yaml

운용 서버 영역 (Main Server, C++)	
작업 항목	상세
TCP 수신 리스너	epoll 기반 비동기 수신 — main_server/src/Listener.cpp
라우팅	station 필드 기반 분기 — main_server/src/Router.cpp
스테이션 핸들러	JSON 파싱·결과 처리 — Station1Handler.cpp / Station2Handler.cpp
결과 관리	검증·통계 집계 — main_server/src/Manager.cpp
DB 연동	MariaDB CRUD — main_server/src/DbManager.cpp
이미지 저장	NG 이미지 파일 저장 — main_server/src/ImageStorage.cpp
GUI 알림	MFC 클라이언트 PostMessage 푸시 — main_server/src/GuiNotifier.cpp
헬스체크	추론서버·학습서버 생존 확인 — main_server/src/HealthChecker.cpp
패킷 프로토콜 정의	main_server/common/Protocol.h

MFC 클라이언트 영역 (Admin GUI, C++)	
작업 항목	상세
페이지 스위칭	mfc_client/ChildView — 탭 전환 프레임
종합 현황 탭	mfc_client/PageHome — 전체 OK/NG, 서버 헬스 상태
①입고 탭	mfc_client/PageStation1 — 카메라 영상, 히트맵, OK/NG
②조립 탭	mfc_client/PageStation2 — YOLO 박스 오버레이, 표면 히트맵
통계/이력 탭	mfc_client/PageStats — 불량률 추이, 파레토 차트, CSV 내보내기
모델 관리 탭	mfc_client/PageModel — 모델 버전 목록, 재학습 트리거
네트워크 수신	mfc_client/NetworkThread — 백그라운드 recv 스레드
사용자 인증	mfc_client/LoginDialog — 로그인 화면, 권한별 메뉴 표시

Arduino / 하드웨어 영역	
작업 항목	상세
①입고 리젝터	서보모터(SG90) 리젝트 + 빨간 LED + 부저 — arduino/Station1Rejecter.ino
②조립 경고기	RGB LED + I2C LCD 16x2 불량 유형 표시 — arduino/Station2Alerter.ino
시리얼 프로토콜	Edge PC와 시리얼(USB) 통신 명령 정의

7.1 영역 간 주요 접점

접점	형식	방향
AI 모델 학습 - AI 추론 서버	모델 파일 배포 (SCP/공유폴더/USB) *.ckpt, *.pt	학습 -> 추론
AI 추론 서버 - 운용 서버	TCP Custom Protocol [4B length][JSON][image?]	추론 -> 운용 (NG만)
운용 서버 - MFC 클라이언트	TCP + PostMessage 실시간 결과 푸시	운용 -> 클라이언트
운용 서버 - MariaDB	SQL CRUD (inspections, bottles, assemblies 등)	운용 <-> DB
추론 서버 - Arduino	Serial (USB) 제어 명령 전송	추론 -> Arduino
운용 서버 - 각 서버	Health-Check TCP Heartbeat 생존 여부 확인	운용 -> 각서버 (주기)

8. GUI 설계 (통합 대시보드)

탭	주요 표시	실시간 요소	상호작용
종합 현황	2단계 OK/NG, 전체 불량률, 가동률, 서버 헬스체크 상태	실시간 수치 업데이트 서버 상태 초록/빨강 표시	스테이션 클릭 시 상세
①입고	카메라 영상, OK/NG, 결함 히트맵	이상 점수 오버레이	수동 판정 버튼
②조립	조립 상태 영상, 캡/라벨 탐지 + 표면 이상 히트맵 YOLO11 바운딩 박스 오버레이	YOLO11 탐지 결과 + PatchCore 이상 점수 오버레이	불량 유형 선택 후 재작업
통계/이력	불량률 추이, 결함 파레토 차트	자동 업데이트	기간/단계 필터, CSV 내보내기
모델 관리	배포된 모델 버전 목록 재학습 트리거 버튼	학습 진행 상태 표시	제품 전환 시 재학습 실행

※ 모델 관리 탭에서 재학습 트리거를 실행하면 AI 학습 서버에 재학습 요청을 전송하고, 완료된 가중치를 자동으로 추론 서버에 배포합니다 (구현 여부는 개발 일정에 따라 결정).

9. DB 스키마 (ERD)

MariaDB를 사용하며, 아래 5개 테이블로 구성됩니다. 사용자 인증, AI 모델 버전 관리, 용기 추적, 전체 검사 결과, 조립 검사 상세를 포함합니다.

users (사용자 계정 및 인증)

컬럼명	타입	설명
id (PK)	INT AUTO_INCREMENT	사용자 ID
employee_id	VARCHAR(50) UNIQUE NOT NULL	사원 ID
username	VARCHAR(50) UNIQUE NOT NULL	로그인 아이디
password_hash	VARCHAR(255) NOT NULL	bcrypt 해시 비밀번호
role	ENUM('admin','operator','viewer')	권한 등급
created_at	DATETIME	계정 생성 시각
last_login_at	DATETIME	마지막 로그인 시각

models (AI 모델 버전 관리)

컬럼명	타입	설명
id (PK)	INT AUTO_INCREMENT	모델 ID
station_id	INT	스테이션 번호 (1 또는 2)
model_type	ENUM('PatchCore','YOLO11')	모델 종류
version	VARCHAR(20)	버전 문자열 (예: v1.2.0)
accuracy	FLOAT	검증 정확도
file_path	VARCHAR(255)	가중치 파일 경로
deployed_at	DATETIME	추론서버 배포 시각
is_active	TINYINT(1)	현재 활성 모델 여부
trained_by (FK)	INT NULL → users.id	학습 등록 사용자 (users 참조)

bottles (용기 상태 추적)

컬럼명	타입	설명
id (PK)	INT AUTO_INCREMENT	용기 ID
bottle_type	VARCHAR(50)	용기 종류 (예: samdasoo_500ml)
batch_no	VARCHAR(50)	배치 번호
status	ENUM('pending','ok','ng')	현재 상태
created_at	DATETIME	생성 시각

inspections (전체 검사 결과)

컬럼명	타입	설명
id (PK)	INT AUTO_INCREMENT	검사 ID
station_id	INT	스테이션 번호 (1 또는 2)
bottle_id (FK)	INT → bottles.id	연결 용기
model_id (FK)	INT → models.id	사용한 모델
timestamp	DATETIME	검사 시각
result	ENUM('ok','ng')	판정 결과
confidence	FLOAT	신뢰도 / 이상 점수
defect_type	VARCHAR(100) NULL	결함 유형 (NULL=정상)
image_path	VARCHAR(255) NULL	NG 이미지 저장 경로 (NG 시에만)
latency_ms	INT	추론 소요 시간 (ms)

assemblies (조립 검사 상세 결과)

컬럼명	타입	설명
id (PK)	INT AUTO_INCREMENT	조립 검사 ID
inspection_id (FK)	INT → inspections.id	연결 검사
bottle_id (FK)	INT → bottles.id	연결 용기
cap_ok	TINYINT(1)	캡 체결 정상 여부
label_ok	TINYINT(1)	라벨 부착 정상 여부
fill_ok	TINYINT(1)	충전량 정상 여부
yolo_detections	JSON	YOLO11 탐지 결과 (박스·클래스 JSON)
patchcore_score	FLOAT	PatchCore 이상 점수 (라벨 표면)
timestamp	DATETIME	검사 시각

9.1 테이블 관계 요약 (ERD)

관계	종류	설명
users → models (trained_by)	1:N	사용자가 여러 모델을 학습 등록할 수 있음
bottles → inspections	1:N	한 용기는 여러 검사를 받을 수 있음
models → inspections	1:N	한 모델은 여러 검사에 사용됨
inspections → assemblies	1:1	조립 검사 결과는 inspections에 1:1 연결
bottles → assemblies	1:N	용기 단위 조립 결과 추적

10. 아두이노 연동 설계

단계	아두이노 역할	필요 부품	통신
①입고	NG 시 서보모터 리젝트 + 빨간 LED + 부저	서보 SG90, LED(R/G), 부저	Serial (USB)
②조립	NG 시 경고등 + LCD에 불량 유형 표시	RGB LED, I2C LCD 16x2	Serial (USB)

11. 개발 일정 (12일)

Day 1~3은 기획 및 환경 구축, Day 4~9는 1단계(입고 검사)와 2단계(조립 검사)를 담당자별 병렬 진행합니다. Day 10~11은 통합 및 최적화, Day 12는 시연입니다.

일차	작업 내용	산출물	비고
Day 1~3	기획·설계: 요구사항 분석, 시스템 아키텍처, DB 스키마, 통신 프로토콜 정의, 기술 선택	시스템 기획서, 환경 구성 (Python, PyTorch, Anomal	
Day 4~9 (병렬)	[1단계 - 입고 검사 담당] · 삼다수 페트병 직접 촬영 + 데이터 증강 · PatchCore 학습 및 성능 검증 · StationRunner #1 + Arduino 리젝터 연동 · 입고 검사 결과 DB 저장 + MFC 탭 구현 [2단계 - 조립 검사 담당] · 정상/불량 조립 촬영 + YOLO11 어노테이션 · YOLO11(구조) + PatchCore(표면) 하이브리드 학습 · StationRunner #2 + Arduino 경고기 연동 · 조립 검사 결과 DB 저장 + MFC 탭 구현	1단계 입고 검사 완성 / 2단계 조립 검사 병행	
Day 10~11	통합: 1단계·2단계 연동 테스트, 운용 서버 통합, 헬스체크 프로세스, 유저 인증	연동 중 통합 통합 테스트 완료, 성능 최적화	
Day 12	최종 시연 + 질의응답	프로젝트 시연 완료	전체

11.1 병렬 개발 구조 (Day 4~9)

구분	1단계 - 입고 검사	2단계 - 조립 검사
AI 모델	PatchCore 학습	YOLO11 + PatchCore 학습
Edge PC	StationRunner #1 + Pylon #1	StationRunner #2 + Pylon #2
Arduino	Station1Rejecter.ino	Station2Alerter.ino
DB	inspections (station=1)	inspections (station=2) + assemblies
MFC 탭	PageStation1	PageStation2
공통 접점	Protocol.h, Packet.py, Config.yaml (운용 서버 IP/포트 공유)	

※ 핵심 전략: Day 4~9에서 1단계·2단계를 병렬로 동시 완성합니다. Day 10~11에 통합 테스트 및 운용 서버 연동을 완료합니다.

12. 상업성 분석 및 포트폴리오 어필

12.1 적용 분야

적용 분야	구체적 시나리오	시장 규모/트렌드
음료 제조	병 검사 → 충전+캡+라벨	HACCP 의무화, 국내 음료 시장 50조+
제약/화장품	용기 검사 → 캡+라벨	GMP 규제, K-뷰티 수출 품질관리
식품 포장	용기 검사 → 포장 완성도	식품안전법 강화, 자동 검사 수요 급증
물류센터	입고 상품 검수 → 피킹 확인	스마트 물류 시장 연 20%+ 성장

12.2 면접 활용 멘트

"본 시스템(Factory)은 음료 생산라인의 입고-공정 전체를 커버하는 통합 비전 QC 시스템입니다. ①단계에서는 CNN 이상탐지(PatchCore)로 삼다수 페트병의 불규칙한 결함을 감지하고, ②단계에서는 YOLO11 객체탐지와 PatchCore 이상탐지를 결합하여 구조적 결함(캡·라벨·충전량)과 표면 품질 결함을 각각 분담 처리합니다. 시스템 인프라(통신/DB/GUI)는 제품이 바뀌어도 재사용 가능하며, 모델만 재학습하면 됩니다. 운용 서버는 4개 서버의 헬스체크를 포함한 완전한 모니터링 시스템을 갖추고 있습니다."

12.3 포트폴리오 어필 포인트

어필 포인트	면접 시 활용
전공경 통합 시스템 설계	"입고부터 조립까지 전 공정을 AI로 관리했습니다"
CNN이 필요한 이유를 설명 가능	"기존 룰 기반으로 못 잡는 불규칙 결함을 이상탐지로, 구조적 결함은 객체탐지로 분담 처리했습니다"
CNN 모델 2종 활용	이상탐지(PatchCore) + 객체탐지(YOLO11) 하이브리드
Full-Stack 기술 스택	C++(Pylon) + Python(AI) + TCP/IP + DB + GUI + Arduino 전부 구현
4-서버 분산 아키텍처	학습서버·추론서버·운용서버 분리 + 헬스체크
데이터 직접 수집	"산업용 카메라로 삼다수 페트병을 직접 촬영/라벨링했습니다"
HACCP/식품안전 도메인 이해	식품/음료/제약 기업 지원 시 규제 키워드로 강력 어필

12.4 확장 가능성

① 로봇팔 연동으로 자동 리젝트 시스템으로 발전. ② Edge AI(Jetson Nano)로 추론 서버 경량화. ③ 실제 MES/ERP 시스템과 API 연동. ④ 화장품/제약 등 다른 용기 라인으로 확장 시 모델 재학습 + 판정 로직 수정으로 대응 가능.